TrialBridge — Project Synopsis

Project Title: TrialBridge: AI-Powered Clinical Trial Patient Matching Platform

1. Introduction

Clinical trials are the backbone of modern medical research. Every new drug, therapy, or treatment protocol has to go through rigorous testing on human participants before it can reach the market. But here is the problem — finding the right patients for the right trials is painfully slow. Studies show that over 80% of clinical trials face delays because of poor patient recruitment, and nearly 30% of enrolled participants end up dropping out because they were not a good fit in the first place. This wastes enormous amounts of time, money, and most importantly, delays life-saving treatments from reaching the people who need them.

The root cause is straightforward. Trial eligibility criteria are written in dense, semi-structured medical text. Patient records are scattered across different systems in different formats. And the people doing the matching — clinical research coordinators — are doing it largely by hand, reading through criteria and cross-referencing patient charts one at a time. It is tedious, error-prone, and does not scale.

TrialBridge was built to solve exactly this problem. It is an AI-powered platform that automates the process of matching patients to clinical trials. The system pulls trial data directly from ClinicalTrials.gov, imports patient records from standardized datasets, and uses a pipeline of LLM-based agents to score and rank patients against trial eligibility criteria. The result is a ranked list of candidates with transparent, explainable scores — so researchers can quickly identify who is most likely to qualify and why.

2. Problem Statement

The current process of matching patients to clinical trials suffers from several fundamental issues:

Manual effort at scale does not work. A single clinical trial might need to screen thousands of patients to enroll a few hundred. Research coordinators reading through eligibility criteria and patient charts one by one simply cannot keep up with the volume, especially when multiple trials are recruiting simultaneously at the same site.

Eligibility criteria are hard to parse programmatically. Trial criteria are written in natural language with medical jargon, abbreviations, nested conditions, and implicit assumptions. A criterion like "patients with uncontrolled Type 2 diabetes (HbA1c > 8.0%) who have not responded to metformin monotherapy" requires understanding of medical terminology, lab value thresholds, and treatment history — things that simple keyword matching cannot handle.

Patient data lives in silos. Electronic health records, lab systems, pharmacy records, and patient-reported outcomes are often in different formats and different systems. Pulling together a complete patient profile for eligibility screening requires manual data aggregation.

There is no feedback loop. When a patient is rejected from a trial, there is rarely a systematic record of why they were rejected, what criteria they failed, and how close they were to qualifying. This means the same patient might get screened for the same trial type repeatedly, or might be overlooked for a trial they actually qualify for.

TrialBridge addresses these issues by creating a unified platform where trial criteria and patient data come together, and an AI pipeline does the heavy lifting of matching, scoring, and explaining the results.

3. Objectives

The project set out to accomplish the following:

- Build a system that can automatically ingest clinical trial data from the ClinicalTrials.gov public API and store it in a structured, queryable format.
- Create a patient data pipeline that can import records from multiple sources (starting with Kaggle datasets and FHIR-compatible data) into a normalized database schema.
- Develop an AI-powered matching engine that uses large language models to understand trial eligibility criteria and identify candidate patients from the database.
- Implement a multi-factor scoring system that evaluates each patient across several dimensions (condition match, age, sex, geography, and health status) to produce a transparent composite score.
- Provide an intuitive visual interface where researchers can select trials, view scored patient lists, inspect individual patient breakdowns, and understand exactly why each patient scored the way they did.
- Support natural language input so that researchers can describe patient criteria in plain English instead of having to work with structured query formats.

4. Scope

What the project covers:

- Automated scraping and storage of recruiting US clinical trials from ClinicalTrials.gov API v2.
- Import of patient records from the Kaggle cardiovascular disease dataset (~~21,000 records) and a FHIR-compatible patient dataset (~~117 records).
- An LLM-based agent pipeline that chains together text-to-SQL generation, root-word search verification, and demographic filtering to match patients to trials.
- A composite scoring engine that evaluates patients across five weighted dimensions with ICD-10 code mapping for condition matching.
- A Streamlit-based analytics dashboard for interactive exploration of results.
- A FastAPI REST API that exposes all matching and scoring functionality as HTTP endpoints.
- Docker containerization for reproducible deployment.
- An audit logging system that records every scoring run for traceability.

What is outside the scope:

- Integration with live EHR systems (Epic, Cerner, etc.) — the project uses static datasets as a proof of concept.
- HIPAA compliance and real patient data handling — all patient data used is synthetic or publicly available.
- Patient-facing features like self-enrollment or consent management.
- Multi-language support for trial criteria (currently English only).

5. Methodology

The project follows a modular, layered architecture where each component can be developed, tested, and scaled independently.

Data Ingestion Layer: The first step was getting trial data into the system. We built an async scraper that hits the ClinicalTrials.gov API v2, fetches studies filtered by status (recruiting), country (US), and study type (interventional), and upserts them into a PostgreSQL database. The scraper handles pagination, rate limiting, and retries gracefully — it can process thousands of trials in a single run without getting throttled or losing data.

For patient data, we built a concurrent CSV importer that can process the Kaggle dataset at high throughput using a worker pool pattern. Each worker maintains its own database session and commits in batches, so the import of 21,000+ records finishes in a matter of minutes rather than hours. The FHIR data path uses a separate SQL-based matching approach that queries patients using the standardized FHIR resource format.

AI Matching Pipeline: The core of the system is a three-stage matching pipeline orchestrated by the TrialMatchOrchestrator:

Stage 1 — Text-to-SQL Agent: When a trial is selected, the system sends the trial's conditions and eligibility criteria to an LLM (via OpenAI or Groq) and asks it to generate a SQL WHERE clause that would identify matching patients. This is not a general-purpose text-to-SQL system — the prompt is carefully engineered to target the specific schema of our patients table, including the tsvector search columns. The generated SQL becomes the first filter to narrow down the candidate pool.

Stage 2 — Root Word Search Agent: The candidates from Stage 1 are verified using PostgreSQL's full-text search with stemming. This catches cases where the LLM-generated SQL was too narrow or too broad. The agent extracts root words from the trial's conditions and searches the patient records using tsvector matching, which handles variations like "diabetic" matching "diabetes" or "hypertensive" matching "hypertension."

Stage 3 — Demographic Filtering: Hard constraints from the trial (age range, sex requirements) are applied as straightforward database filters. Patients outside these ranges are either excluded entirely or penalized in their score depending on how far outside the range they fall.

Scoring Engine: Each patient that makes it through the matching pipeline gets scored across five dimensions:

- Condition Match (30 points): Uses ICD-10 code mapping to determine how well the patient's conditions align with what the trial is studying. We load a 14MB ICD-10 reference dataset and map patient conditions to standardized codes, then check overlap with the trial's target conditions.
- Age Fit (20 points): Scored on a curve — patients at the center of the trial's age range get full marks, with scores tapering off toward the edges.
- Sex Alignment (15 points): Binary match against the trial's sex requirement.
- Geographic Proximity (15 points): Checks if the patient's state matches any of the trial's site locations within the US.
- Health Status (20 points): Maps the patient's self-reported general health to a numerical score, with adjustments for BMI, smoking status, and other health indicators when available.

The final composite score is the sum of all five dimensions, normalized to a 0-100 scale. Patients who hit hard exclusion criteria (conditions explicitly excluded by the trial, completely outside the age range, wrong sex for a sex-specific trial) are flagged as excluded with clear reasons, rather than just getting a low score.

Natural Language Interface: For researchers who do not want to browse through trial listings, the system supports free-text criteria input. The user can type something like "elderly women with uncontrolled diabetes and obesity" and the system sends this to the LLM for structured extraction — pulling out conditions, age range, sex filter, BMI constraints, and smoking status. These extracted parameters are then used to construct a virtual trial profile, and the scoring pipeline runs against it just like it would for a real trial. The system shows the researcher what it extracted from their input so they can verify the interpretation.

6. Technology Stack

Component	Technology	Justification

Backend Framework	FastAPI (Python)	Async-native, automatic API documentation, Pydantic validation
Database	PostgreSQL	Full-text search support, JSONB for flexible trial payloads, mature async drivers
ORM	SQLAlchemy 2.0 (async)	Type-safe queries, async session support, Alembic migration integration
LLM Integration	OpenAI, Groq	Access to multiple model providers with a single API pattern
Dashboard	Streamlit	Rapid prototyping of data-heavy interfaces, built-in charting
Visualization	Plotly, Matplotlib	Interactive charts, radar diagrams, and histograms
Containerization	Docker, Docker Compose	Reproducible builds, easy deployment
Package Management	uv	Fast dependency resolution, lockfile support
Logging	Logfire	Structured logging with production observability

7. Expected Outcomes

- A working platform that can match patients to clinical trials with transparent, multi-factor scoring in under 10 seconds per request.
- Demonstrably better matching accuracy compared to simple keyword-based approaches, thanks to ICD-10 code mapping and LLM-powered criteria understanding.
- A clean, interactive dashboard that makes it easy for non-technical researchers to explore results without writing queries or reading JSON.
- An extensible architecture where new data sources (like real EHR feeds), new scoring dimensions, or new LLM providers can be plugged in without rewriting the core system.
- An audit trail that records every scoring decision for reproducibility and compliance purposes.

8. Conclusion

TrialBridge demonstrates that the clinical trial patient matching problem — traditionally solved with manual effort and simple rule engines — can be significantly improved by combining modern LLM capabilities with structured medical data and a well-designed scoring framework. The platform is not trying to replace clinical judgment; it is trying to surface the most promising candidates so that researchers can focus their time on the patients who are most likely to qualify, rather than sifting through thousands of records hoping to find a match. The modular architecture means the system can grow from a proof-of-concept with synthetic data into a production tool integrated with real healthcare systems, without needing to be rebuilt from the ground up.